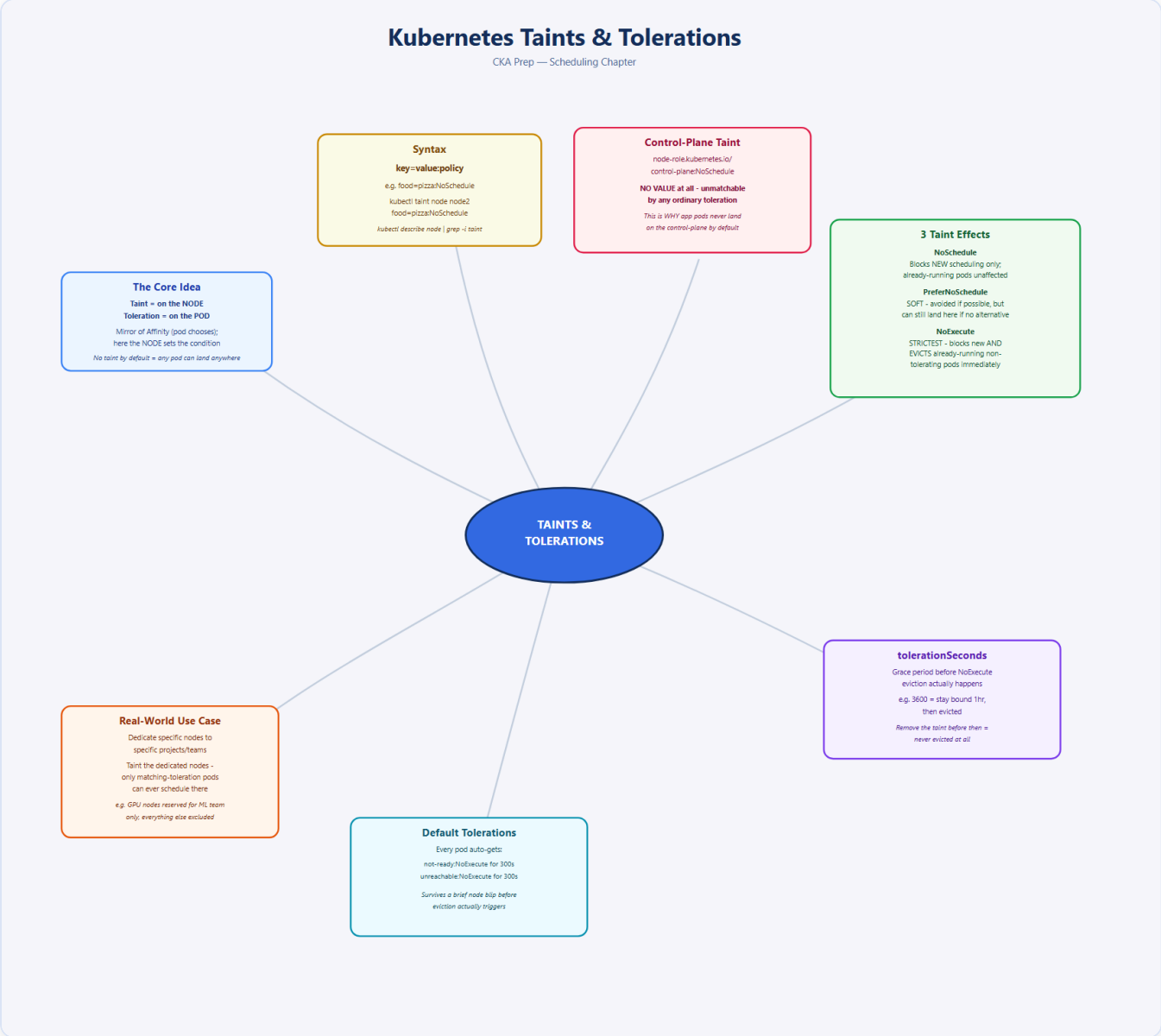
KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 07

Kubernetes Taints and Tolerations

Topic: Scheduling Restrictions | Core Kubernetes | 2026-09-16

Quick Recall — Mind Map



The Core Idea: Nodes Reject Pods by Default Rule, Not Accept Them

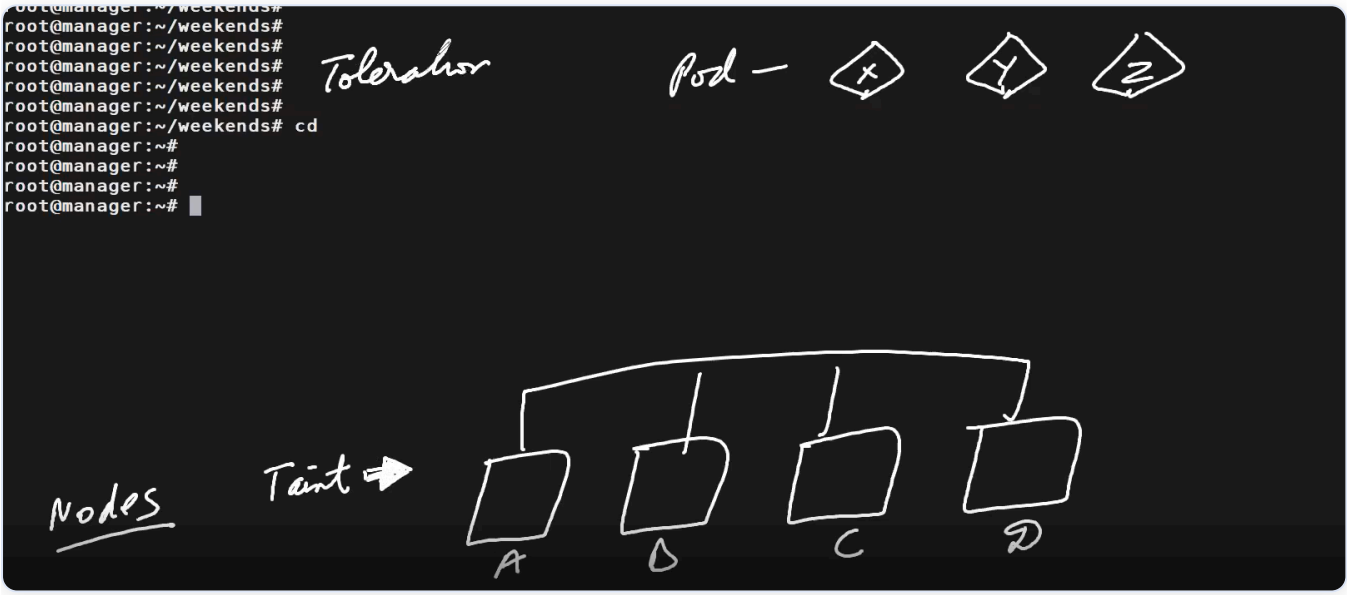
Taints and Tolerations mean: **nodes accept only certain kinds of pods.**

- **Taint is always applied to a NODE.**

- **Toleration is always applied to a POD.**

This is the mirror image of Node Affinity/NodeSelector (the pod's perspective, choosing a node) — here, the **node** is the one setting the condition, and a pod must explicitly "agree" to it via a toleration to be allowed on.

> **Memory aid**
>
> "Taint" literally means a bad smell/contamination. **By default, no worker node has any taint** — which is exactly why, by default, any pod can be scheduled onto any worker node. A taint is what a node uses to actively repel pods that don't explicitly tolerate it.



Worked Example — One Taint, One Condition

Cluster: worker nodes A, B, C, D. Pods X, Y, Z want to be deployed.

Suppose Node D is configured with a taint: "*I will only run pods that like pizza.*"

- When pods X, Y, Z are considered for Node D, Node D effectively asks each one: "*do you like pizza?*"
- If a pod **doesn't** carry the matching toleration, it will **not** run on Node D — it gets scheduled on A, B, or C instead.
- If a pod **does** carry a toleration for that exact condition, it's allowed to run on D too.

Second example, same idea: Node D's condition is "*do you wake up at 5 AM?*" Among X, Y, Z, only pod Y has a toleration saying "yes." **Only pod Y** can be scheduled onto Node D.

Worked Example — Multiple Nodes, Multiple Taints

- Node A: taint = `red` Node B: taint = `blue`

- Node C: taint = `red` Node D: taint = `blue`

No pod can run on any of these nodes by default. Now suppose pods X and Y carry a toleration for `blue` → they can run on **Node B or D**. Pods Z and W carry a toleration for `red` → they can run on **Node A or C**.

> **Real-world pattern**
>
> This is exactly how you **dedicate certain nodes to certain projects/workloads**: taint the dedicated nodes, and only pods carrying the matching toleration can land there.

Defining a Taint — the Syntax

```
SYNTAX
taint = key=value:policy
```

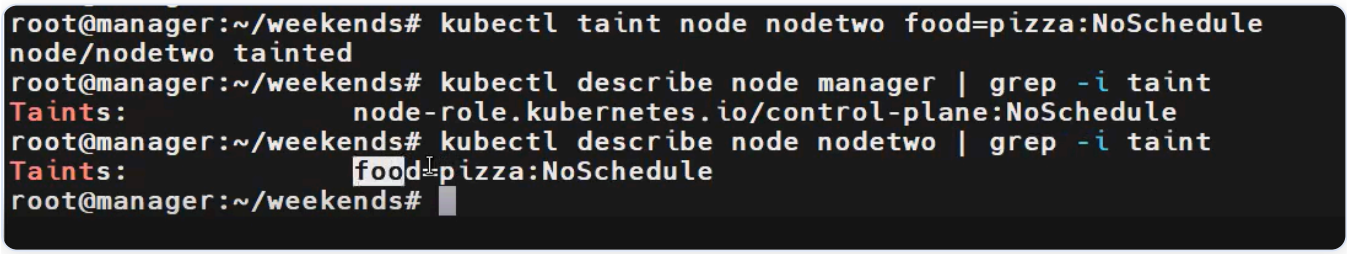
Example: `food=pizza:NoSchedule` — key `food`, value `pizza`, policy `NoSchedule`.

Command to Taint a Node

```
BASH
kubectl taint node node2 food=pizza:NoSchedule
# node/node2 tainted

kubectl describe node node2 | grep -i taint
# Taints:    food=pizza:NoSchedule
```

⚠ The Control-Plane Node's Taint — a Special Case




```
BASH
kubectl describe node manager | grep -i taint
# Taints:    node-role.kubernetes.io/control-plane:NoSchedule
```

No value at all

Compare this to `key=value:policy` — the manager's taint has **no value** (`node-role.kubernetes.io/control-plane` straight to `:NoSchedule`). This means the condition is effectively unmatchable by any normal pod toleration — **no ordinary pod can ever fulfill it**, which is exactly why regular application pods never get scheduled onto the control-plane node by default.

Adding a Toleration to a Pod

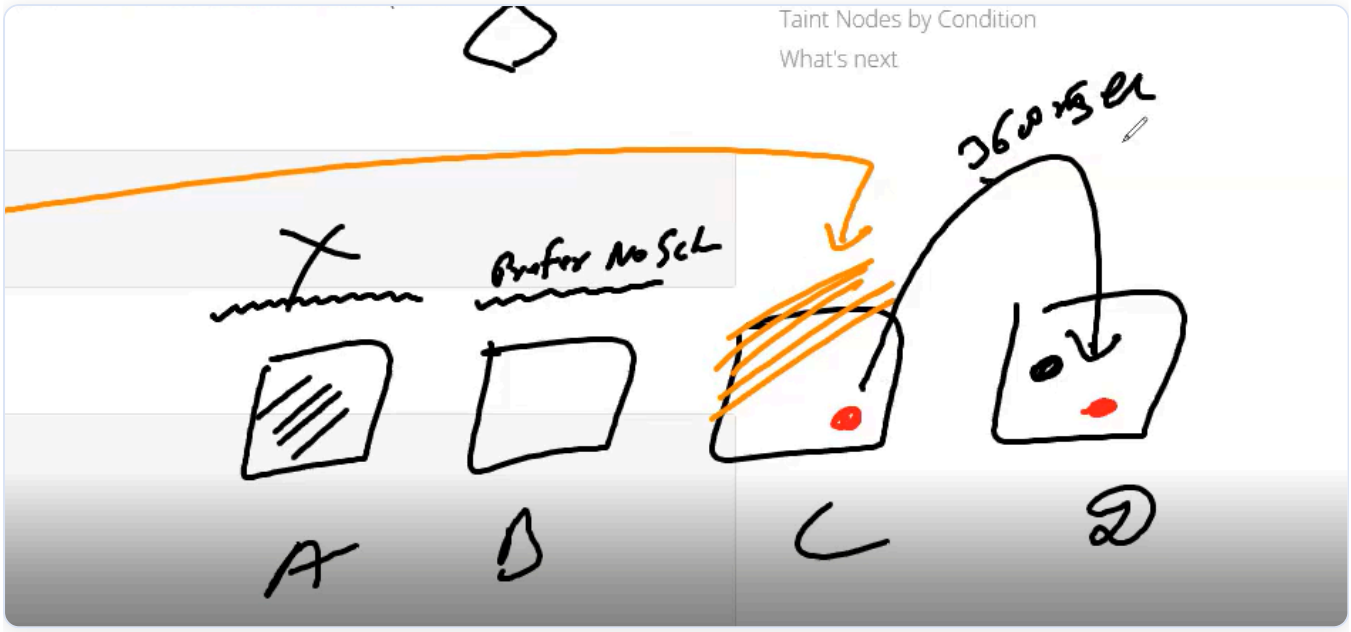
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp1
  labels: { app: myapp }
spec:
  containers:
    - name: ngnix-container
      image: nginx
  tolerations:
    - key: "color"
      operator: "Equal"
      value: "pink"
      effect: "NoSchedule"
```

For this pod to actually land on a specific tainted node, the toleration's `key` / `value` / `effect` must exactly match the taint that node was given.

Taint Effects — 3 Policies, Different Strictness

Effect	Meaning
NoSchedule	Hard rule for *new* scheduling — a pod without a matching toleration will never be scheduled here. Pods already running (from before the taint existed) are unaffected.
PreferNoSchedule	A **soft** limit. The scheduler tries to avoid placing non-tolerating pods here, but if there's genuinely nowhere else to run, it *can* still land here anyway.
NoExecute	The strictest. Not only blocks new scheduling — it actively **evicts pods already running** on the node if they don't have a matching toleration, the moment the taint is applied.

NoExecute in Action — the Eviction Scenario



Suppose Node C has **no taints**, and a pod is already happily running there. Now, some time later, you apply a `NoExecute` taint to Node C. Since the pod has no toleration for the new taint, Kubernetes will **evict it and move it to another machine** — `NoExecute` doesn't just gate future scheduling, it actively kicks out what's already there.

tolerationSeconds — a Grace Period Before Eviction

```yaml
tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoExecute"
    tolerationSeconds: 3600
```

The pod stays bound to that node for 3600 seconds after a matching `NoExecute` taint appears, then gets evicted. If the taint is **removed** before that window elapses, the pod is never evicted at all.

Practical Examples From the Session

YAML

```yaml
# A Burstable-QoS pod that tolerates the food=pizza:NoSchedule taint from earlier
apiVersion: v1
kind: Pod
metadata: { name: burst }
spec:
  containers:
    - name: qos-demo-ctr
      image: nginx
      resources:
        limits: { memory: "128Mi", cpu: "100m" }
        requests: { memory: "100Mi", cpu: "50m" }
  tolerations:
    - key: "food"
      operator: "Equal"
      value: "pizza"
      effect: "NoSchedule"
```

BASH

```bash
kubectl describe pod burst | grep -A3 "Tolerations"
# Node-Selectors:  <none>
# Tolerations:     food=pizza:NoSchedule
#                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
#                  node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
```

Default tolerations on every pod

Kubernetes automatically adds the `not-ready` / `unreachable` (300s) tolerations to every pod, so it briefly tolerates a node going not-ready/unreachable before being evicted, rather than getting yanked instantly on a transient blip.

Why This Matters — the Real-World Use Case

Dedicating specific nodes to specific projects/teams/workloads. Taint the dedicated nodes with a project-specific key/value, and only pods explicitly configured with the matching toleration can ever be scheduled there — e.g. GPU-only nodes reserved strictly for an ML team, or compliance-sensitive nodes reserved for a specific regulated workload.

What I Practiced

- [ ] Tainted a worker node and confirmed it via `kubectl describe node | grep -i taint`.
- [ ] Compared the control-plane node's built-in taint (no value) against a normal `key=value:policy` taint, and reasoned through why that makes it unmatchable by any ordinary pod toleration.
- [ ] Wrote a Pod with a matching `tolerations` block and confirmed via `kubectl describe pod` that both my explicit toleration and Kubernetes' default tolerations were present.
- [ ] Reasoned through all 3 effects (NoSchedule hard-blocks new scheduling only, PreferNoSchedule is a soft preference, NoExecute actively evicts).
- [ ] Understood the `tolerationSeconds` grace-period mechanic for NoExecute.

Merged into the master reference notes: k8s-master-notes.md §6 (Scheduling & Autoscaling) — new "Taints & Tolerations — Deep Dive" addendum.